

Operating Systems Function Prototypes

Processes

```
pid_t fork(void);
int execve(const char *filename, char *const argv[], char *const envp[]);
pid_t wait(int *status);
pid_t waitpid(pid_t pid, int *status, int options);
void _exit(int status);
        WIFEXITED(); // macro
        WEXITSTATUS(); // macro
```

Threads

```
int pthread_create(pthread_t *thread, pthread_attr_t *attr,
        void *(*start_routine)(void *), void *arg);
void pthread_exit(void *retval);
int pthread_join(pthread_t th, void **thread_return);
pthread_t pthread_self(void);
int pthread_detach(pthread_t thread);
```

Mutexes/Locks

```
pthread_mutex_t fastmutex = PTHREAD_MUTEX_INITIALIZER;
int pthread_mutex_init(pthread_mutex_t *mutex,
        const pthread_mutexattr_t *mutexattr);
int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_trylock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Condition Variables

```
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
int pthread_cond_destroy(pthread_cond_t *cond);
```

Semaphores

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_wait(sem_t *sem);
int sem_trywait(sem_t *sem);
int sem_post(sem_t *sem);
int sem_getvalue(sem_t *sem, int *sval);
int sem_destroy(sem_t *sem);
```

File System

```
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
        flags: O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_APPEND
        mode: S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP,
S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
int close(int fd);
ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);
off_t lseek(int fd, off_t offset, int whence);
        whence: SEEK_SET, SEEK_CUR, SEEK_END
DIR *opendir(const char *name);
struct dirent *readdir(DIR *dirp);
        struct dirent {
                ino_t      d_ino;          /* Inode number */
                off_t      d_off;          /* Not an offset; see below */
                unsigned short d_reclen;    /* Length of this record */
                unsigned char d_type;       /* Type of file */
                char        d_name[256];    /* Null-terminated filename */
        };
d_type:
        DT_BLK      This is a block device.
        DT_CHR      This is a character device.
        DT_DIR      This is a directory.
        DT_FIFO     This is a named pipe (FIFO).
        DT_LNK      This is a symbolic link.
```

```

        DT_REG      This is a regular file.
        DT_SOCK     This is a UNIX domain socket.
        DT_UNKNOWN  The file type could not be determined.
int closedir(DIR *dirp);
int stat(const char *pathname, struct stat *statbuf);
    struct stat {
        dev_t      st_dev;          /* ID of device containing file */
        ino_t      st_ino;          /* Inode number */
        mode_t     st_mode;         /* File type and mode */
        nlink_t    st_nlink;        /* Number of hard links */
        uid_t      st_uid;          /* User ID of owner */
        gid_t      st_gid;          /* Group ID of owner */
        dev_t      st_rdev;         /* Device ID (if special file) */
        off_t      st_size;         /* Total size, in bytes */
        blksize_t  st_blksize;      /* Block size for filesystem I/O */
        blkcnt_t   st_blocks;       /* Number of 512B blocks allocated */
        struct timespec st_atim;    /* Time of last access */
        struct timespec st_mtim;    /* Time of last modification */
        struct timespec st_ctim;    /* Time of last status change */
    };

```

Signals

```

int kill(pid_t pid, int sig);
int pause(void);
int sigsuspend(const sigset_t *mask);
int sigprocmask(int how, const sigset_t *set, sigset_t *oldset);
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
    struct sigaction {
        void      (*sa_handler)(int);
        void      (*sa_sigaction)(int, siginfo_t *, void *);
        sigset_t  sa_mask;
        int       sa_flags;
        void      (*sa_restorer)(void);
    };
int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signum);
int sigdelset(sigset_t *set, int signum);
sighandler_t signal(int signum, sighandler_t handler);
    typedef void (*sighandler_t)(int);

```